

DAY 8 TECHNICAL REPORT

Searching Algorithms & Algorithmic Efficiency Optimization

Trainee:	Benatic Rithan B	Institution:	KIT, Coimbatore
Mentor:	Rathees G	Department:	CSE
Date:	17 June 2026	Status:	COMPLETED SUCCESSFULLY

1. EXECUTIVE SUMMARY

This technical report documents the design, implementation, and mathematical evaluation of sequential and divide-and-conquer search methodologies engineered during Day 8 of the Python Backend Development Internship. Moving beyond the direct $O(1)$ key-based hash map lookups established during the Day 6 inventory data structure selection, the operational objective of this session focused on comparing asymptotic runtimes when querying elements sequentially versus logarithmically. To benchmark these concepts under realistic enterprise constraints, a Student Search System was developed to track structural execution steps, verify collection state requirements, and isolate processing bottlenecks when querying data records by non-primary unique identifiers.

2. DATA STRUCTURE SELECTION & COMPLEXITY ARCHITECTURE

To optimize record lookup routines within high-volume backend data layers, search operations are categorized by their structural prerequisites and asymptotic growth rates:

- **Linear Search ($O(n)$):** Implemented as a sequential traversal mechanism. It evaluates data nodes from the baseline index 0 through $n-1$ systematically. While structurally flexible because it requires no pre-sorting processing, its execution time scales linearly with dataset growth, making it inefficient for deep database queries.
- **Binary Search ($O(\log n)$):** Implemented as a divide-and-conquer interval reduction system. It samples the exact median index of a sorted workspace, discarding half of the remaining searchable dataset at every execution loop. This minimizes the worst-case step growth logarithmically, though it introduces a strict data-ordering prerequisite.

3. STRUCTURAL MAPPING MATRIX

The following table maps the algorithmic properties evaluated during system testing:

Collection Type	System Implementation Role	Core Structural Feature	Primary Operational Advantage
Linear Search	Secondary/Unsorted Query Engine	Sequential Element Scan	High flexibility; zero structural setup overhead.
Binary Search	Primary Record Lookup Engine	Divide-and-Conquer Interval Halving	Logarithmic scaling efficiency for large, pre-sorted data pools.

4. TECHNICAL CODE IMPLEMENTATION

The code segment below showcases the integration of structural step-tracking within an object-oriented manager to benchmark both paradigms while processing a structured collection of records:

```
class StudentSearchSystem:
    def __init__(self):
        # Collection array populated with dictionary structures sorted by ID
        self.students = [
            {"id": 101, "name": "Alice", "gpa": 3.8},
            {"id": 105, "name": "Bob", "gpa": 3.4},
            {"id": 110, "name": "Charlie", "gpa": 3.9},
            {"id": 115, "name": "Diana", "gpa": 3.2},
            {"id": 120, "name": "Evan", "gpa": 3.7}
        ]

    def search_by_id_linear(self, target_id):
        """Sequentially scans the collection array. Works on any data ordering."""
        steps = 0
        for student in self.students:
            steps += 1
            if student["id"] == target_id:
                return student, steps
        return None, steps

    def search_by_id_binary(self, target_id):
        """Divides the search space in half each step. Requires sorted data."""
        steps = 0
        low = 0
        high = len(self.students) - 1

        while low <= high:
            steps += 1
            mid = (low + high) // 2
```

```

        mid_val = self.students[mid]["id"]

        if mid_val == target_id:
            return self.students[mid], steps
        elif mid_val < target_id:
            low = mid + 1
        else:
            high = mid - 1

    return None, steps

```

5. WORKSPACE STUDENT STATE MOCKUPS

PRIMARY SORTED CATALOG (SELF.STUDENTS STATE REPRESENTATION)

```

[
    {"id": 101, "name": "Alice", "gpa": 3.8},
    {"id": 105, "name": "Bob", "gpa": 3.4},
    {"id": 110, "name": "Charlie", "gpa": 3.9},
    {"id": 115, "name": "Diana", "gpa": 3.2},
    {"id": 120, "name": "Evan", "gpa": 3.7}
]

```

WORST-CASE SCENARIO SIMULATION (TARGET ID: 120)

Linear Search Execution Path:

- Step 1: Evaluates ID 101 → No Match.
- Step 2: Evaluates ID 105 → No Match.
- Step 3: Evaluates ID 110 → No Match.
- Step 4: Evaluates ID 115 → No Match.
- Step 5: Evaluates ID 120 → **Match Confirmed.**

Total Efficiency Cost: 5 Steps

Binary Search Execution Path:

- Step 1: Evaluates Median Index 2 (ID 110). Target 120 > 110. Shifts range right (low = 3).
- Step 2: Evaluates Median Index 4 (ID 120). Target 120 == 120. **Match Confirmed.**

Total Efficiency Cost: 2 Steps

6. LEARNING OUTCOMES

- **Structural Selection Mastery:** Evaluated runtime space-time tradeoffs to choose appropriate search strategies depending on collection sorting states.

- **Prerequisite Enforcement:** Verified that higher-order algorithms like Binary Search depend strictly on collection sortedness states, whereas Linear Search acts as a stable fallback for arbitrary, unsorted collections.
- **Asymptotic Performance Tracking:** Demonstrated practically how logarithmic step growth $O(\log n)$ preserves processor bandwidth compared to linear scale growth $O(n)$ when parsing heavy enterprise records.
- **Complex Object Aggregation:** Constructed nested collection schemes (lists containing mixed dictionaries of strings, floats, and integer fields) to handle and query structured enterprise records efficiently.